



PAUTA TAREA 1

Pregunta 1

Alternativa 1: definición con función de transición

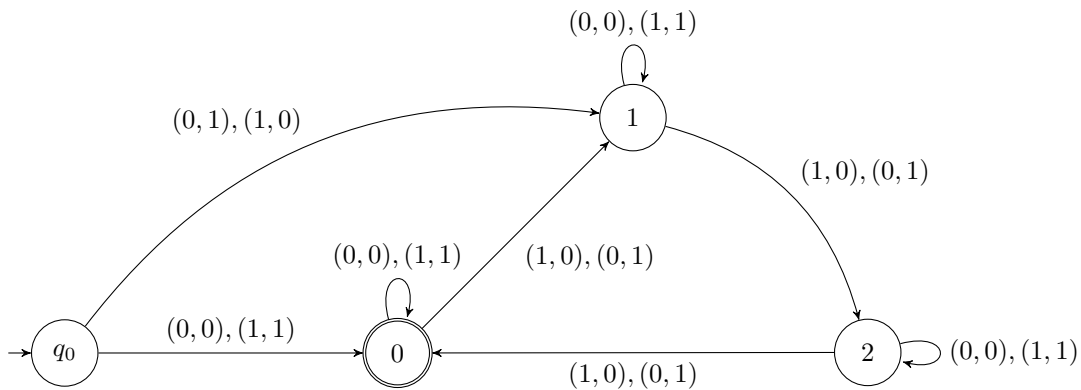
Definimos el automata $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$:

$$\begin{aligned} Q &= \{q_0, 0, 1, 2\} \\ \Sigma &= \{0, 1\}^2 \\ q_0 &= q_0 \\ F &= \{0\} \end{aligned}$$

Y definimos la función de transición de la siguiente manera:

$$\begin{aligned} \delta(q_0, (c, d)) &= \begin{cases} 0 & \text{si } c = d, \\ 1 & \text{si } c \neq d. \end{cases} \\ \delta(k, (c, d)) &= \begin{cases} k & \text{si } c = d \\ (k + 1) \bmod 3 & \text{si } c \neq d. \end{cases} \end{aligned}$$

Alternativa 2: definición con diagrama



Demostración de Correctitud:

Para demostrar correctitud del automata propuesto bastaría con explicar en palabras como el automata es capaz de generar el lenguaje pedido cumpliendo con las dos condiciones necesarias para que una palabra

pertenezca al lenguaje. En particular las condiciones que debe cumplir las palabras aceptadas por el automata son: $n > 0$ y $H(a_1 \cdots a_n, b_1 \cdots b_n) \bmod 3 = 0$. Demostraremos a continuación que el automata definido cumple ambas condiciones:

1) Notamos que el automata cuenta con un estado inicial auxiliar q_0 que se encarga de filtrar las palabras vacías dado que solo transiciona si el largo de la palabra cuenta con al menos una tupla.

2) Para demostrar que cumple con $H(a_1 \cdots a_n, b_1 \cdots b_n) \bmod 3 = 0$, demostraremos con inducción estructural que el automata cumple con la siguiente propiedad: dada una entrada $x = (a_1, b_1) \cdots (a_n, b_n)$ $\forall i \in \{1, \cdots, n\}$ se cumple que $p_i = H(a_1 \cdots a_i, b_1 \cdots b_i) \bmod 3$. En otras palabras la propiedad establece que una vez terminada la i -ésima ejecución el automata guarda en su estado el mod 3 del acumulado de hamming.

Caso base: Si $i = 0$ tenemos que $p_0 = q_0$ sabemos por el inciso 1 que si $x = \epsilon$ esta se queda en el estado q_0 y el automata lo rechazaría. Luego, si trabajamos con una entrada no vacía para $i = 1$ notamos que de q_0 existen dos posibles transiciones si $a_1 = b_1$ obtenemos $H(a_1, b_1) = 0$ y por lo tanto $H(a_1, b_1) \bmod 3 = 0 = p_1$. Y si $a_1 \neq b_1$ obtenemos $H(a_1, b_1) = 1$ y por lo tanto $H(a_1, b_1) \bmod 3 = 1 = p_1$. Con lo anterior concluimos que se una vez finalizada la primera ejecución el estado guarda el mod 3 del acumulado de la distancia de hamming de la palabra.

Hipótesis inductiva: Vamos a suponer que dada una entrada arbitraria $x > n$ se cumple la propiedad para todo $i > 1$

Tesis inductiva: A continuación buscamos demostrar que la hipótesis se cumple para la ejecución $i + 1$.

Sobre la entrada $x = (a_1, b_1) \cdots (a_n, b_n)$ sabemos que una vez finalizada la i -ésima ejecución se ha procesado $x' = (a_1, b_1) \cdots (a_i, b_i)$ y por hipótesis inductiva sabemos que p_i guarda el registro de la operación mod 3 sobre el acumulado de la distancia de hamming que se ha leído hasta este punto. Ahora, tocaría procesar la tupla (a_{i+1}, b_{i+1}) y es importante notar lo siguiente, si tenemos la palabra $x'(a_{i+1}, b_{i+1})$ y le calculamos su distancia de hamming esta será: $H(a_1 \cdots a_i a_{i+1}, b_1 \cdots b_i b_{i+1})$ lo cual cumple con la siguiente igualdad $H(a_1 \cdots a_i a_{i+1}, b_1 \cdots b_i b_{i+1}) = H(a_1 \cdots a_i, b_1 \cdots b_i) + H(a_{i+1}, b_{i+1})$ (se deja como propuesta la demostración de esta igualdad). Luego, si aplicamos la operación mod 3 sobre la expresión obtenemos $(H(a_1 \cdots a_i, b_1 \cdots b_i) + H(a_{i+1}, b_{i+1})) \bmod 3 = (H(a_1 \cdots a_i, b_1 \cdots b_i) \bmod 3 + H(a_{i+1}, b_{i+1}) \bmod 3) \bmod 3$ (propiedad de aritmética modular). Finalmente, aplicando la hipótesis inductiva obtenemos: $(p_i + H(a_{i+1}, b_{i+1}) \bmod 3) \bmod 3$. Para esta última expresión existen los siguientes casos:

$$\begin{aligned} \text{Si } a_{i+1} = b_{i+1} &\rightarrow H(a_{i+1}, b_{i+1}) = 0 \bmod 3 = 0 \\ (p_i + H(a_{i+1}, b_{i+1}) \bmod 3) \bmod 3 &= (p_i + 0) \bmod 3 = p_i = p_{i+1} \end{aligned}$$

Notamos, que este primer caso está explícitamente definido en el automata en los loops de cada estado al leer tuplas con valores iguales por lo que el mod 3 de la distancia de Hamming acumulada permanece igual y queda guardado en el estado de la ejecución p_{i+1} . Luego, el siguiente caso sería:

$$\begin{aligned} \text{Si } a_{i+1} \neq b_{i+1} &\rightarrow H(a_{i+1}, b_{i+1}) = 1 \bmod 3 = 1 \\ (p_i + H(a_{i+1}, b_{i+1}) \bmod 3) \bmod 3 &= (p_i + 1) \bmod 3 = p_{i+1} \end{aligned}$$

Notamos, que se cumple la transición definida en el automata. En particular este caso cubre cuando se tiene un acumulado de por ejemplo 0, 1 o 2 y se procesa una tupla con caracteres distintos.

Finalmente, podemos asegurar que el automata mantiene en sus estados un registro de la operación mod 3 sobre el acumulado de la distancia de hamming.

Luego, dado que el automata toma una entrada w y solo la acepta si cumple ambas propiedades demostradas

podemos asegurar que $\mathcal{A}$ acepta $w \leftrightarrow w \in L$ dejando demostrado que el automata define el lenguaje propuesto.

Distribución de puntaje

- 1 punto por cumplir con aceptar palabras de $n > 0$.
- 1 punto por manejar los casos de tuplas con caracteres iguales y distintos.
- 1 punto por cumplir con mantener registro de la operación mod 3 sobre el acumulado de hamming.
- 1 punto por explicar la correctitud de su construcción.

Pregunta 2

Existen 2 posibles soluciones para esta pregunta.

Alternativa 1: Definición de autómeta

Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómeta finito determinista (DFA) tal que $\mathcal{L}(A) = L$. A partir del cual construimos un nuevo DFA $A^{*-} = (Q^{*-}, \Sigma, \delta^{*-}, q_0, F^{*-})$ tal que:

- $Q^{*-} = \{q_0\} \cup (Q \times \Sigma \times \Sigma)$
- $\delta^{*-}(q_0, c) = (\delta(q_0, c), c, c)$
- $\delta^{*-}((p, c_1, c_2), c) = (\delta(p, c), c_1, c)$
- $F^{*-} = \{(p, c, c) \mid p \in F \wedge c \in \Sigma\}$

Ahora demostraremos que $\mathcal{L}(A^{*-}) = L^{*-}$ por doble contención.

$$\mathcal{L}(A^{*-}) \subseteq L^{*-}$$

Sea $u = a_1 \dots a_n \in \mathcal{L}(A^{*-})$, por construcción tenemos que la ejecución del autómeta A^{*-} sobre u es:

$$\rho : q_0 \xrightarrow{a_1} (q_1, a_1, a_1) \xrightarrow{a_2} (q_2, a_1, a_2) \xrightarrow{a_3} \dots \xrightarrow{a_n} (q_n, a_1, a_n)$$

tal que $\delta(q_i, a_{i+1}) = q_{i+1}$. Dado que ρ es una ejecución de aceptación entonces se cumple que $a_1 = a_n$ y $q_n \in F$. Sea entonces la ejecución ρ' de A sobre u , tal que:

$$\rho' : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} \dots \xrightarrow{a_n} q_n$$

Una ejecución de aceptación, por lo que se cumple que $u \in \mathcal{L}(A)$ y $a_1 = a_n$. Por lo tanto, $u \in L(A^{*-})$.

$$\mathcal{L}(A^{*-}) \supseteq L^{*-}$$

Sea $u = a_1 \dots a_n \in L(A^{*-})$, por lo que se cumple que $u \in \mathcal{L}(A)$ y que $a_1 = a_n$. Por lo que existe la ejecución de aceptación ρ sobre u tal que:

$$\rho : q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} \dots \xrightarrow{a_n} q_n$$

tal que $\delta(q_i, a_{i+1}) = q_{i+1}$ y $q_n \in F$. Dado lo anterior, entonces sabemos que por construcción, existe una ejecución de aceptación ρ' del autómeta A^{*-} sobre u tal que:

$$\rho' : q_0 \xrightarrow{a_1} (q_1, a_1, a_1) \xrightarrow{a_2} (q_2, a_1, a_2) \xrightarrow{a_3} \dots \xrightarrow{a_n} (q_n, a_1, a_n)$$

Y como $q_n \in F$ y $a_1 = a_n$, entonces se cumple que $(q_n, a_1, a_n) \in F^{*-}$. Por lo tanto, $u \in \mathcal{L}(A)$.

Dado lo anterior, la distribución de puntaje es la siguiente:

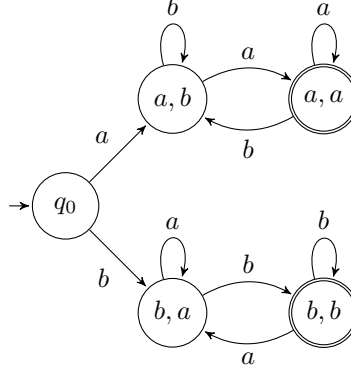
- **(1 punto)** Por definir correctamente $Q^{*-} = \{q_0\} \cup Q \times \Sigma \times \Sigma$.
- **(1 punto)** Por definir correctamente las transiciones δ^{*-} .
- **(1 punto)** Por definir correctamente $F^{*-} = \{(p, c, c) \mid p \in F \wedge c \in \Sigma\}$.
- **(1 punto)** Por demostrar correctamente que $\mathcal{L}(A^{*-}) = L^{*-}$ con doble contención.

Alternativa 2: Construcción de automata

Suponga que L es un lenguaje regular y sea el lenguaje R , tal que:

$$R = \{w = c \cdot v \cdot c \mid c \in \Sigma \wedge v \in \Sigma^*\}$$

Es decir, el lenguaje de las palabras que empiezan y terminan con la misma letra. Para demostrar que R es regular creamos el autómata $\mathcal{A}'$, que acepta palabras que empiezan y terminan con la misma letra. La siguiente figura define el automata $\mathcal{A}'$ y hace lo pedido:



El automata creado funciona de la siguiente manera: comienza en el estado inicial q_0 , desde el cual avanza a los estados (a, b) o (b, a) , dependiendo de cual es la primera letra. Si es la a avanza a (a, b) , mientras que, si la primera letra es b avanza a (b, a) . Los siguientes “camino” funcionan de manera homologa desde los estados (a, b) y (b, a) , solo diferenciandose en que los las letras son las contrarias, por lo que ejemplificaremos con el estado (a, b) . Dado que a este estado se llega con la letra inicial a , si la siguiente letra es b se mantiene en el estado. En caso contrario, si la siguiente letra es a , el autómata avanza al estado (a, a) , el cual es un estado de aceptación. Si hay una siguiente letra, si es a , se mantiene en el mismo estado, en caso contrario, la siguiente letra es b , entonces vuelve al estado (a, b) , el cual no es un estado de aceptación, de tal forma que acepta palabras que empiezan con a y terminan con a . En el caso desde el estado (b, a) , funciona similar, con el estado de aceptación (b, b) , de tal forma que acepta palabras que empiezan y terminan con b .

Dado que R es regular y L es regular, entonces tenemos que la intersección de los lenguajes $L \cap R$ es regular, esto según lo que se vio en clases. Ahora demostraremos que $L^{*-} = L \cap R$.

$$L^{*-} \subseteq L \cap R$$

Sea $u \in L^{*-}$, por lo que sabemos que, por definición del lenguaje L^{*-} , $u \in L$ y así también que existe $c \in \Sigma$ y existe $v \in \Sigma^*$ tal que $u = c \cdot v \cdot c$. Lo cual implica que, entonces $u \in R$, y por lo tanto, que $u \in L \cap R$.

$$L^{*-} \supseteq L \cap R$$

Sea $u \in L \cap R$. Por definición, se cumple que $u \in L$ y $u \in R$, al pertenecer a ambos lenguajes implica que existe un $c \in \Sigma$, y un $v \in \Sigma^*$ tal que $u = c \cdot v \cdot c$. Por definición, entonces tenemos que $u \in L^{*-}$.

En conclusión, se cumple que $L^{*-} = L \cap R$, y por consiguiente, L^{*-} es un lenguaje regular.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por definir correctamente el lenguaje R .
- **(1 punto)** Por construir correctamente el automata $\mathcal{A}'$, y explicarlo.
- **(1 punto)** Por utilizar correctamente la propiedad de intersección de lenguajes regulares.
- **(1 punto)** Por demostrar correctamente $L^{*-} = L \cap R$.